Федеральное государственное бюджетное образовательное учреждение высшего образования
«Херсонский государственный педагогический университет»

Институт информационных технологий

Кафедра программной инженерии и информационных технологий

ПРАКТИЧЕСКАЯ РАБОТА
по дисциплине:
«Процессы и профессиональная практика программной инженерии»

на тему:
«Практическая работа 9.
Управление событиями пользовательского интерфейса»

Выполнил:
Студент гр.: 12-25РПм
Трифонов Д.С.

Проверил
Преподаватель:
Алешов Е.В.

Оценка:_____
Дата: 13.11.2026

Херсон, 2026

ПРАКТИЧЕСКАЯ РАБОТА

Тема: Управление событиями пользовательского интерфейса. Цель работы — изучить обработку событий в GUI на основе паттерна «Наблюдатель», принцип восходящего уведомления уровней (UNP) в PCMEF, а также применение паттернов «Команда», «Цепочка обязанностей» и организацию модели данных таблицы (аналог JTable / TableModel) на примере учебного сценария управления исходящими сообщениями.

1. Ход выполнения работы

1. Изучен теоретический материал практической работы: модель событий Swing/MVC, роли издателя и подписчика (ActionListener), размещение логики фильтрации/сортировки в модели таблицы, механизмы Undo/Redo на основе стеков команд, сценарии Create New Message и удаления исходящего сообщения.

2. Для практической демонстрации на вымышленных данных реализовано минимальное приложение на Python (библиотека tkinter) в каталоге app/. Программа моделирует основной GUI исходящих сообщений: таблица, кнопки Insert / Delete / Toggle Filter / Undo / Redo / Exit, журнал событий и подтверждения действий.

3. В коде выделены аналоги ключевых ролей методички: EventBus — издатель событий (Observer/UNP), MessageStore — модель данных (PMessageTableModel / PDisplayList), представление таблицы подписывается на событие model_changed; операции Insert/Delete оформлены как объекты Command и сохраняются в стеках Undo/Redo.

4. Использованы вымышленные данные исходящих сообщений (отправители Иванов, Петрова, Сидоров, Козлова; партнёры ООО «Альфа», ЗАО «Бета», ИП «Гамма»). Выполнены сценарии: просмотр списка, фильтрация по партнёру, добавление сообщения, отмена через Undo, валидация пустой темы (диалог Unable to Send), подтверждение удаления.

5. Запуск программы: python app/main.py (стандартная библиотека tkinter, дополнительные пакеты не требуются).

2. Результаты (скриншоты)

Ниже приведены результаты работы программы на вымышленных данных. Скриншоты иллюстрируют обработку UI-событий и реакцию представления на изменения модели.

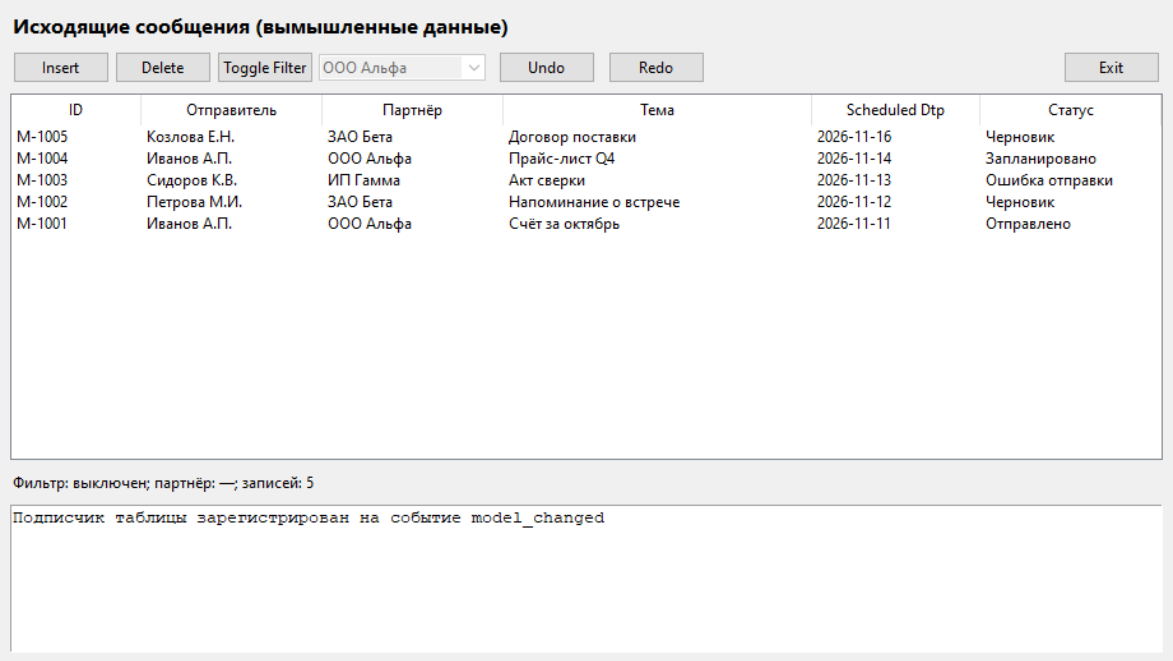


Рисунок 1 — Главное окно: таблица исходящих сообщений, подписчик зарегистрирован на model_changed

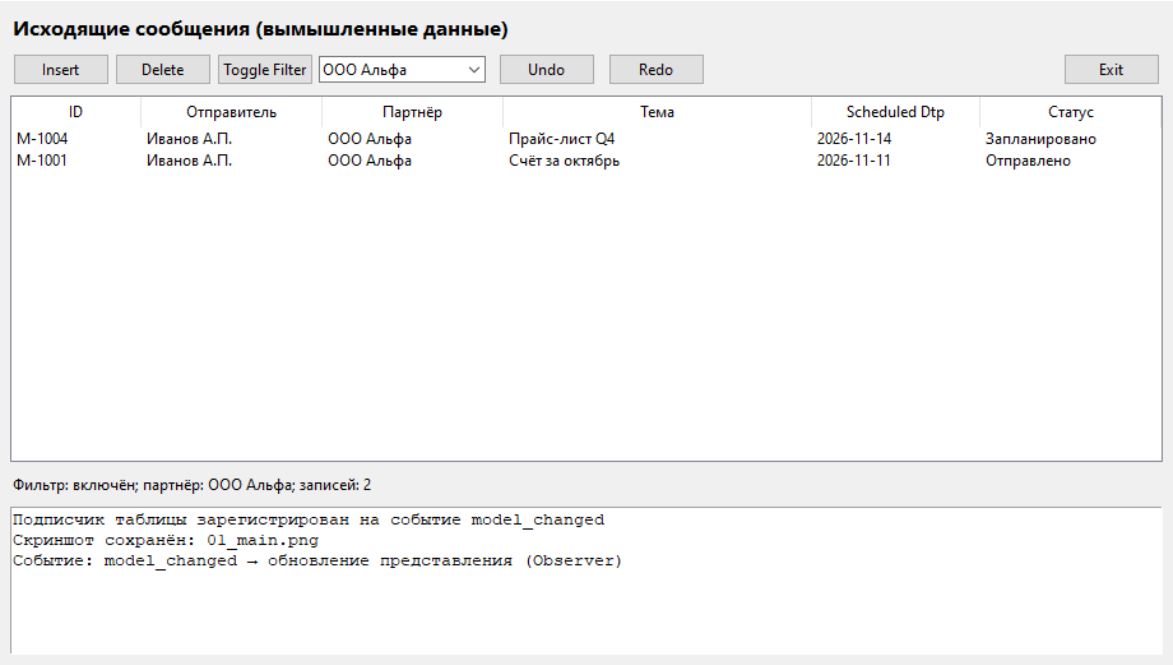


Рисунок 2 — Событие Toggle Filter: фильтрация по партнёру «ООО Альфа» (видимы 2 записи)

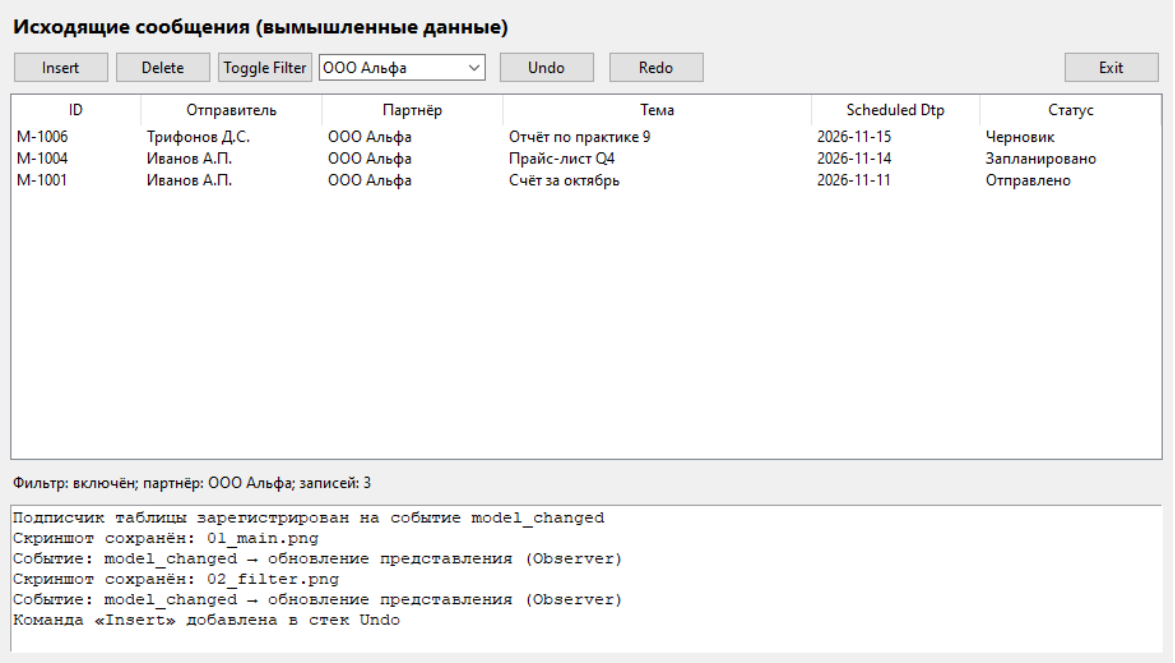


Рисунок 3 — После Insert: добавлено сообщение «Отчёт по практике 9», команда помещена в стек Undo

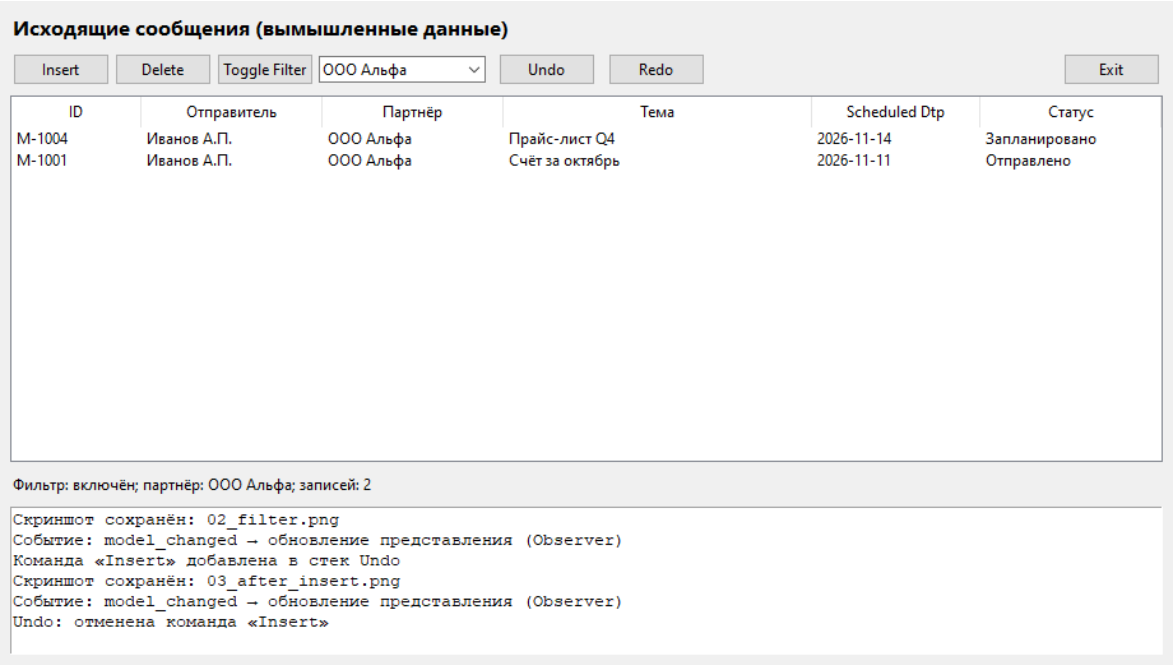


Рисунок 4 — Выполнение Undo: отмена Insert, восстановление предыдущего состояния модели

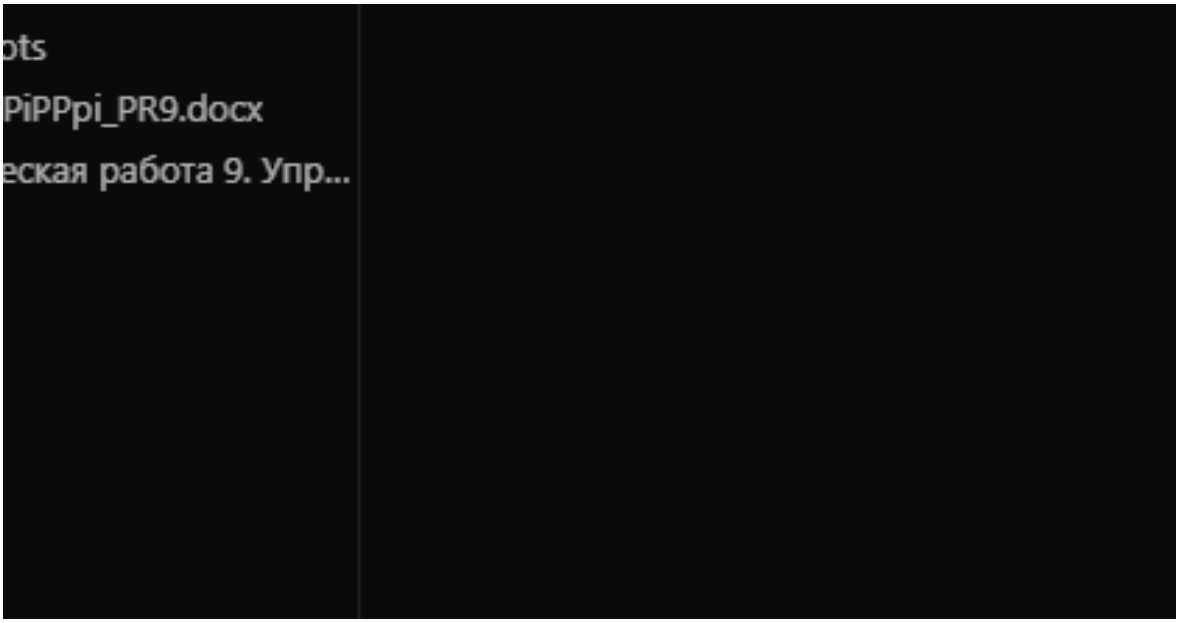


Рисунок 5 — Диалог Create New Message (создание нового исходящего сообщения)

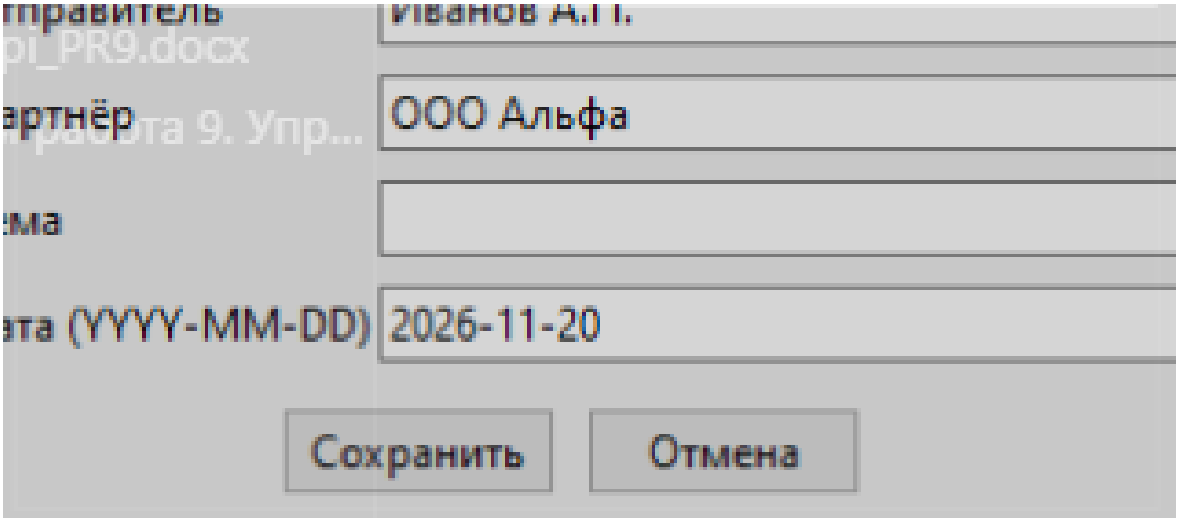


Рисунок 6 — Диалог Unable to Send при нарушении валидации обязательных полей

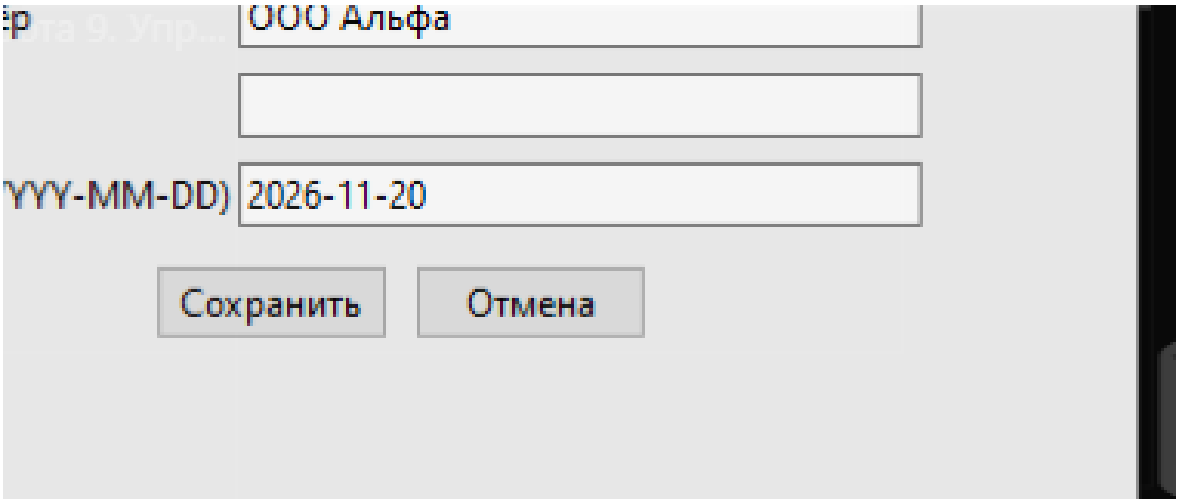


Рисунок 7 — Диалог подтверждения удаления исходящего сообщения

3. Ответы на контрольные вопросы

1. Какую роль играет обработка событий в PCMEF-шаблоне с точки зрения принципа UNP и какого уровня архитектуры она касается?

Обработка событий обеспечивает восходящее уведомление уровней (UNP): источник события на уровне presentation извещает заинтересованных подписчиков о действиях пользователя. В PCMEF это касается уровня presentation; в более сложных случаях отдельный класс-подписчик может располагаться на уровне control и слушать издателя presentation.

2. В чем заключается взаимодействие между объектом-источником события (издателем) и объектами-подписчиками, и как происходит регистрация на примере ActionListener?

Издатель генерирует событие при действии пользователя и уведомляет всех зарегистрированных подписчиков. Подписчик реализует интерфейс приёмника (например, ActionListener) и метод actionPerformed(ActionEvent). Регистрация выполняется вызовом addActionListener(...): подписчик может быть отдельным классом, внутренним или анонимным внутренним классом.

3. Как организованы сортировка и фильтрация данных в JTable на примере PMessageTableModel, PDisplayList и PDisplayData?

Запросы фильтрации идут от PWindow к PMessageTableModel. Модель делегирует фильтрацию списку PDisplayList, который отбрасывает неподходящие PDisplayData. После фильтрации JTable запрашивает у модели данные по строкам/столбцам и отображает уже отфильтрованное подмножество. Сортировка также размещается в ответственности модели.

4. Как паттерн «Наблюдатель» реализуется при взаимодействии JTable с JTableModel и TableCellEditor?

JTable подписывается на модель через addTableModelListener. При изменении данных модель уведомляет слушателей (tableChanged), и таблица перерисовывается. Редактирование ячейки идёт через TableCellEditor: после editingStopped значение передаётся в модель (setValueAt), модель снова уведомляет наблюдателей — таблица обновляется.

5. Как работает паттерн «Цепочка обязанностей» на примере контекстной справки Windows («What's this?») и каковы его недостатки?

Запрос справки передаётся по цепочке компонентов GUI (от текущего элемента вверх по иерархии контейнеров), пока кто-то не обработает запрос. Недостатки: неочевидность конечного обработчика, сложность отладки, риск «проглатывания» запроса или дублирования ответственности при длинной цепочке.

6. В чем отличие паттерна «Команда» от «Цепочки обязанностей» и как реализуется Undo/Redo?

«Команда» инкапсулирует действие как объект с методами выполнения/отмены, а не ищет обработчик в цепочке. Для Undo/Redo выполненные команды помещаются в стек; undo() извлекает команду и вызывает отмену, redo() использует второй стек для повторного применения.

7. Какова последовательность создания заместителя нового исходящего сообщения до открытия окна редактирования (MDataMapper, OID, EIdentityMap)?

До показа PMessageDetailWindow формируется заместитель сообщения: MDataMapper создаёт OID и регистрирует объект в EIdentityMap. Сообщение помечается как изменённое

для последующего цикла сохранения. Только после этого открывается окно деталей для редактирования.

8. Как синхронизируются кэш и БД при сохранении сообщения из PMessageDetailWindow?

При сохранении MDataMapper синхронизирует кэш EIdentityMap с БД. Для нового сообщения нельзя полагаться на временный OID: messageID получают из БД, затем сообщение перерегистрируют в EIdentityMap уже с реальными OID и messageID, завершая корректировку.

9. В чем различия программируемого клиента и клиента-браузера?

Программируемый клиент выполняется локально и может использовать ресурсы машины пользователя. Клиент-браузер зависит от сервера: данные и инструкции отображения загружаются по сети и представляются через Web-технологии.

10. На какие три группы делятся GUI-компоненты Swing и какую функцию выполняют менеджеры расположения?

Компоненты Swing делятся на контейнеры, меню и элементы управления. Менеджеры расположения (layout managers) определяют размещение дочерних компонентов внутри контейнера, задавая геометрию интерфейса.

4. Вывод

В ходе практической работы изучены принципы управления событиями пользовательского интерфейса: паттерн «Наблюдатель» для связи модели и представления, роль ActionListener/издателя-подписчика, организация фильтрации в модели таблицы и реализация Undo/Redo через паттерн «Команда». На вымышленных данных продемонстрированы сценарии фильтрации, создания и удаления исходящих сообщений, а также реакция UI на события изменения модели. Контрольные вопросы проработаны на основе материала методички.

Программа: app/main.py (Python, tkinter). Дата выполнения: 13.11.2026.